Custom Hooks in React with TypeScript

What Is a Custom Hook?

A **custom hook** is just a JavaScript function that:

1. Starts with the word `use`
2. Can call other React hooks inside it (`useState` , `useEffect` , etc.)
3. Returns whatever values the component needs

That's it. There is no special React API for creating custom hooks. They are just functions with a naming convention.

Why do they exist?

React hooks (`useState` , `useEffect` , etc.) can only be called inside a React component or another hook. If you find yourself copy-pasting the same hook logic across multiple components, you extract it into a custom hook — exactly like you'd extract repeated logic into a function.

> **Mental model:** Custom hooks are to hook logic what utility functions are to regular logic.

Table of Contents

---

1. `useToggle`

Problem it solves (Without a Custom Hook)

Imagine you have three components: a Modal, a Dropdown, and an Accordion. Each one needs to show or hide itself based on a boolean.

Without a custom hook:

```
// Modal.tsx
function Modal() {
  const [isOpen, setIsOpen] = useState(false);
  const toggle = () => setIsOpen(prev => !prev);

  return (
```

```
    <div>
      <button onClick={toggle}>Toggle Modal</button>
      {isOpen && <div className="modal">Modal Content</div>}
    </div>
  );
}


// Dropdown.tsx
function Dropdown() {
  const [isOpen, setIsOpen] = useState(false);  // same line
  const toggle = () => setIsOpen(prev => !prev); // same line

  return (
    <div>
      <button onClick={toggle}>Toggle Dropdown</button>
      {isOpen && <div className="dropdown">Options here</div>}
    </div>
  );
}


// Accordion.tsx
function Accordion() {
  const [isOpen, setIsOpen] = useState(false);  // same line again
  const toggle = () => setIsOpen(prev => !prev); // same line again

  return (
    <div>
      <button onClick={toggle}>Toggle Section</button>
      {isOpen && <p>Accordion content</p>}
    </div>
  );
}
```

What's the problem?

- These two lines are identical in every component:

```
const [isOpen, setIsOpen] = useState(false);
const toggle = () => setIsOpen(prev => !prev);
```

- If you want to change the toggle logic (e.g., add logging, add a callback), you must change it in every component.
- The toggle logic is **not a UI concern** — it's a **behavior concern**. It belongs outside the component.

The fix: Extract this into a custom hook.

The Concept

When should you use it?

- Any time you have open/closed, visible/hidden, on/off state.
- When you need the same toggle pattern in multiple components.

When should you NOT use it?

- When you only have one component that needs a boolean — `useState` is simpler and fine.
- When the boolean logic is more complex than a simple flip (e.g., it depends on other conditions).

Step-by-Step Implementation

Step 1 — The Simplest Version (No TypeScript Yet)

```
// hooks/useToggle.js
function useToggle() {
  const [value, setValue] = useState(false);

  function toggle() {
    setValue(prev => !prev);
  }

  return [value, toggle];
}
```

Every line explained:

```
function useToggle() {
```

This is a regular JavaScript function. The name starts with `use` — this is a React convention. React uses this naming to enforce the Rules of Hooks (you'll see warnings if you violate them).

```
const [value, setValue] = useState(false);
```

We're using `useState` inside our custom hook. This is allowed. The hook "owns" this state. Every component that calls `useToggle` gets its own **independent copy** of this state.

```
function toggle() {
  setValue(prev => !prev);
}
```

This is the flip function. We use `prev => !prev` (a **functional update**) instead of `setValue(!value)`. `prev` is always the current, up-to-date state. `!prev` flips it.

```
return [value, toggle];
```

We return an **array** with two items. The component will use array destructuring to name them whatever it wants:

```
const [isOpen, toggle] = useToggle();
const [isVisible, toggleVisibility] = useToggle();
const [isDarkMode, toggleDarkMode] = useToggle();
```

Step 2 — Add Optional Initial Value

```
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  function toggle() {
    setValue(prev => !prev);
  }

  return [value, toggle];
}
```

What changed?

- `initialValue = false` is a default parameter. If you call `useToggle()`, it uses `false`. If you call `useToggle(true)`, it starts as `true`.
- We pass `initialValue` to `useState` instead of hardcoding `false`.

Step 3 — Add TypeScript

```
// hooks/useToggle.ts
import { useState } from "react";

function useToggle(initialValue: boolean = false): [boolean, () => void] {
  const [value, setValue] = useState(initialValue);

  function toggle(): void {
    setValue(prev => !prev);
  }

  return [value, toggle];
}

export default useToggle;
```

Every type explained:

`initialValue: boolean = false`

- `: boolean` — this parameter must be a boolean
- `= false` — the default value (JavaScript feature, not TypeScript)

> What would happen if we removed `: boolean` ? TypeScript would infer it as `boolean` anyway from the default value `false` . It's optional here, but explicit is clearer.

`: [boolean, () => void]` — The Tuple Return Type

This is the **return type** of the function. It's a **Tuple type**.

What is a Tuple?

A tuple is an array where the number of elements is fixed and each position has a specific type:

```
// Regular array — all elements the same type
const numbers: number[] = [1, 2, 3];

// Tuple — each position has a defined type
const pair: [boolean, () => void] = [false, () => {}];
//           pos 0    pos 1
```

Why `() => void` ? `()` — the function takes no parameters. `void` — the function returns nothing.

Why do we need an explicit return type?

Without it, TypeScript infers:

```
return [value, toggle];
// TypeScript infers: (boolean | (() => void))[]
// position 0 could be a function, position 1 could be a boolean ← WRONG
```

By specifying `[boolean, () => void]` :

```
// Position 0 is always boolean. Position 1 is always a function. ← CORRECT
```

> **Interview point:** When a function returns an array, TypeScript defaults to inferring a wider array type. Use an explicit tuple return type (or `as const`) to narrow it.

Alternative with `as const` :

```
return [value, toggle] as const;
// Return type: readonly [boolean, () => void]
```

Step 4 — Final Implementation

```
// hooks/useToggle.ts
import { useState } from "react";

function useToggle(initialValue: boolean = false): [boolean, () => void] {
```

```
  const [value, setValue] = useState<boolean>(initialValue);

  function toggle(): void {
    setValue(prev => !prev);
  }

  return [value, toggle];
}

export default useToggle;
```

> Note: `useState<boolean>` is optional here — TypeScript infers `boolean` from `initialValue`. But it makes the code more self-documenting.

Step 5 — How a Component Uses It

```
// components/Modal.tsx
import useToggle from "../hooks/useToggle";

function Modal() {
  const [isOpen, toggle] = useToggle(false);

  return (
    <div>
      <button onClick={toggle}>
        {isOpen ? "Close Modal" : "Open Modal"}
      </button>

      {isOpen && (
        <div className="modal">
          <h2>Modal Content</h2>
          <button onClick={toggle}>Close</button>
        </div>
      )}
    </div>
  );
}


// components/Accordion.tsx
import useToggle from "../hooks/useToggle";

function Accordion({ title, content }: { title: string; content: string }) {
  const [isExpanded, toggle] = useToggle(false);

  return (
    <div>
      <button onClick={toggle}>
```

```
        {title} {isExpanded ? "▲" : "▼"}
      </button>
      {isExpanded && <p>{content}</p>}
    </div>
  );
}
```

Notice: the hook returns `[value, toggle]`, but the component names them `[isOpen, toggle]` and `[isExpanded, toggle]`. Array destructuring lets you choose any name.

More usage examples:

```
const [isOpen, toggle] = useToggle(false);         // modal
const [isDropdownOpen, toggleDropdown] = useToggle(); // dropdown
const [isNavOpen, toggleNav] = useToggle(false);   // mobile nav
const [isDarkMode, toggleDarkMode] = useToggle(true); // dark mode (starts on)
```

Step 6 — Execution Flow

```
Component renders
        ↓
useToggle(false) is called
        ↓
useState(false) runs — state initialized to false
        ↓
toggle function is created
        ↓
[false, toggle] returned to component
        ↓
User clicks button → toggle() called
        ↓
setValue(prev => !prev) — prev = false, result = true
        ↓
React schedules re-render
        ↓
Component re-renders
        ↓
useState returns existing state = true (initial value ignored)
        ↓
[true, toggle] returned — UI updates
```

Important: On every re-render, `useToggle(false)` is called again, but `useState` does **not** re-initialize to `false`. React remembers the current state across renders. The `false` initial value is only used on the **first** render.

Common Mistakes — `useToggle`

Mistake 1: Using current value instead of `prev`

```
// ❌ Wrong
const toggle = () => setValue(!value);

// ✅ Correct
const toggle = () => setValue(prev => !prev);
```

Why is the wrong version dangerous?

React can batch state updates. `value` inside the closure might be stale (an old snapshot). The functional update `prev => !prev` always uses the **latest** state from React's queue.

Example of the problem:

```
// Toggle called twice rapidly:
toggle(); // reads value = false, sets to true
toggle(); // reads value = false (stale!), sets to true again
// Expected: false → true → false
// Actual:   false → true → true (bug!)
```

Mistake 2: Not naming the hook with `use`

```
// ❌ Wrong
function toggleHook() { ... }

// ✅ Correct
function useToggle() { ... }
```

React's linting rules enforce the `use` prefix. Without it, violations of Rules of Hooks go undetected.

Recap: `useToggle`

Concept	What you learned
Custom hook structure	A function starting with `use` that calls React hooks
State is per-component	Multiple components share the same logic, not the same state
Functional state update	`prev => !prev` is safer than `!value`
Tuple return type	`[boolean, () => void]` fixes TypeScript's inference
Array destructuring	Lets callers name values freely

---

2. `useLocalStorage`

Problem it solves (Without a Custom Hook)

Imagine you want to remember a user's theme preference across page refreshes.

```
// Without custom hook
function App() {
  const [theme, setTheme] = useState("light");

  // Load from localStorage on mount
  useEffect(() => {
    const saved = localStorage.getItem("theme");
    if (saved) {
      setTheme(saved);
    }
  }, []);

  // Save to localStorage on every change
  useEffect(() => {
    localStorage.setItem("theme", theme);
  }, [theme]);

  return <div className={theme}>...</div>;
}
```

Now you also want to remember the username:

```
function ProfilePage() {
  const [username, setUsername] = useState("");

  useEffect(() => {
    const saved = localStorage.getItem("username");
    if (saved) setUsername(saved);
  }, []);

  useEffect(() => {
    localStorage.setItem("username", username);
  }, [username]);
}
```

Problems:

- 6+ lines repeated every time you want persistence.
- No handling of corrupted data — could crash your app.
- Reading from localStorage on every render (can be avoided with lazy initializer).

The fix: A `useLocalStorage` hook that behaves exactly like `useState` but persists automatically.

The Concept

What does `useLocalStorage` do? It's `useState` with automatic persistence to localStorage.

What does it accept?

- A `key` (the localStorage key name, e.g., `"username"`)
- An `initialValue` (used only if there's nothing in localStorage yet)

What does it return? The same two things as `useState`: the current value and a setter function.

When to use it?

- When you want state that survives page refreshes.

When NOT to use it?

- For temporary UI state (open/closed, hover) — no need to persist.
- For sensitive data (passwords, tokens) — localStorage is not secure.
- For large objects — localStorage has a ~5 MB limit and reading/writing is synchronous.

Step-by-Step Implementation

Step 1 — The Core Idea: Lazy Initializer

Before writing anything, understand this crucial React concept.

`useState` also accepts a **function** as the initial value:

```
const [value, setValue] = useState(() => {
  // This function runs ONLY on the first render
  return computeExpensiveValue();
});
```

This is called a **lazy initializer**.

Why does this matter?

```
// ❌ Runs on EVERY render (wasteful)
useState(localStorage.getItem("username"))

// ✅ Runs ONLY on first render
useState(() => localStorage.getItem("username"))
```

`localStorage.getItem` is synchronous and blocks the main thread. There's no reason to call it on every render when React only uses the result once (on first render).

> **Rule:** Whenever the initial value computation is expensive (reading from storage, parsing, etc.), use a lazy initializer.

Step 2 — Simplest Version (No TypeScript, No Error Handling)

```
// hooks/useLocalStorage.js
import { useState, useEffect } from "react";

function useLocalStorage(key, initialValue) {
  // Lazy initializer — runs only on first render
  const [storedValue, setStoredValue] = useState(() => {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : initialValue;
  });

  // Sync to localStorage whenever storedValue changes
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(storedValue));
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}
```

Why JSON?

localStorage can only store strings:

```
// Without JSON — WRONG
localStorage.setItem("user", { name: "Vijeth" });
// Stored as: "[object Object]"

// With JSON.stringify — CORRECT
localStorage.setItem("user", JSON.stringify({ name: "Vijeth" }));
// Stored as: '{"name":"Vijeth"}'

// Reading back with JSON.parse — CORRECT
JSON.parse('{"name":"Vijeth"}');
// Returns: { name: "Vijeth" }
```

Step 3 — Add Error Handling

What if localStorage contains corrupted data?

`JSON.parse("{'broken json")` throws a `SyntaxError`. If not caught, this crashes your entire React app.

```
const [storedValue, setStoredValue] = useState(() => {
  try {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) : initialValue;
  } catch (error) {
    console.warn(`useLocalStorage: error reading key "${key}"`, error);
    return initialValue; // Fall back gracefully
```

```
  }
});
```

Why `console.warn` and not `throw`?

A corrupted cache entry should not crash the user's app. We log the warning so developers can see it, then fall back to `initialValue` — the app continues working.

Step 4 — Add TypeScript and Generics

Why we need generics here:

Without generics:

```
function useLocalStorage(key: string, initialValue: any) { ... }
```

With `any`, TypeScript loses all type information:

```
const [name, setName] = useLocalStorage("username", "");
setName(123); // TypeScript doesn't complain — but this is a bug!
```

What are generics?

A generic is a way to write a function that works with any type while staying type-safe. Think of it as a variable for types:

```
// Generic: T holds a type
function identity<T>(value: T): T {
  return value;
}

identity("hello");  // T = string
identity(42);       // T = number
identity(true);     // T = boolean
```

TypeScript **infers** `T` from the argument — you rarely write it explicitly.

Applying generics to `useLocalStorage`:

```
function useLocalStorage<T>(
  key: string,
  initialValue: T
): [T, React.Dispatch<React.SetStateAction<T>>] {
```

- `<T>` — declares a type variable
- `initialValue: T` — the initial value is of type `T`; TypeScript fills in `T` from this
- **Return** `[T, React.Dispatch<React.SetStateAction<T>>]` — a tuple

What is `React.Dispatch<React.SetStateAction<T>>`?

This is the exact type of the setter that `useState` returns. Let's decode it:

`React.SetStateAction<T>` means: either a new value of type `T`, OR a function that takes the previous `T` and returns a new `T`:

```
// Both are valid SetStateAction<string>:
setName("Vijeth");              // Direct value
setName(prev => prev + "!");    // Functional update
```

`React.Dispatch<React.SetStateAction<T>>` is: "a function you can call with either a new value or an updater function."

> **Why the complex type?** We return `setStoredValue` directly from `useState`. The type must match exactly what `useState` returns. **Can TypeScript infer it?** Yes — if you omit the return type annotation, TypeScript infers it correctly. But being explicit makes the API clear.

Step 5 — Final Implementation

```
// hooks/useLocalStorage.ts
import { useState, useEffect } from "react";

function useLocalStorage<T>(
  key: string,
  initialValue: T
): [T, React.Dispatch<React.SetStateAction<T>>] {

  // Lazy initializer: only runs on first render
  const [storedValue, setStoredValue] = useState<T>(() => {
    try {
      const item = localStorage.getItem(key);
      return item ? (JSON.parse(item) as T) : initialValue;
    } catch (error) {
      console.warn(`useLocalStorage: error reading key "${key}"`, error);
      return initialValue;
    }
  });

  // Sync to localStorage when storedValue changes
  useEffect(() => {
    try {
      localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.warn(`useLocalStorage: error writing key "${key}"`, error);
    }
  }, [key, storedValue]);

  return [storedValue, setStoredValue];
}
```

```
export default useLocalStorage;
```

Note on `JSON.parse(item) as T`: `JSON.parse()` returns `any`. We use `as T` to tell TypeScript: "trust me, this is type `T`." We use it because we stored the value ourselves (with `JSON.stringify`), so we know it's the right shape.

How TypeScript infers `T` for different usages:

```
// T = string
const [name, setName] = useLocalStorage("username", "");

// T = number
const [age, setAge] = useLocalStorage("age", 25);

// T = boolean
const [isDarkMode, setIsDarkMode] = useLocalStorage("darkMode", false);

// T = { name: string; age: number }
const [user, setUser] = useLocalStorage("user", { name: "", age: 0 });

// Explicit (redundant — TypeScript already knows)
useLocalStorage<string>("username", "");
```

Step 6 — How a Component Uses It

```
// components/Settings.tsx
import useLocalStorage from "../hooks/useLocalStorage";

function Settings() {
  const [username, setUsername] = useLocalStorage("username", "");
  const [isDarkMode, setIsDarkMode] = useLocalStorage("darkMode", false);

  return (
    <div className={isDarkMode ? "dark" : "light"}>
      <input
        value={username}
        onChange={(e) => setUsername(e.target.value)}
        placeholder="Enter your name"
      />
      <button onClick={() => setIsDarkMode(prev => !prev)}>
        Toggle Theme
      </button>
      <p>Hello, {username}!</p>
    </div>
  );
}
```

This behaves exactly like `useState`. The component doesn't need to know about `localStorage` at all.

Step 7 — Execution Flow

First render:

```
Component renders
        ↓
useLocalStorage("username", "") is called
        ↓
useState(lazyInitializer) runs
        ↓
  Lazy initializer executes:
  localStorage.getItem("username")
        ↓
  Data exists?
  ↙                    ↘
Yes                     No
  ↓                       ↓
JSON.parse(item)        use ""  (initialValue)
  ↓                       ↓
  state = "Vijeth"        state = ""
        ↓
Component renders with current value
```

When `setName("Vijeth")` is called:

```
setName("Vijeth") called
        ↓
React state updates: storedValue = "Vijeth"
        ↓
Component re-renders (lazy initializer NOT called again)
        ↓
useEffect fires (storedValue changed)
        ↓
localStorage.setItem("username", '"Vijeth"')
        ↓
Next page load: lazy initializer reads '"Vijeth"' and restores it
```

Common Mistakes — `useLocalStorage`

```
// ❌ Reading localStorage on every render (no lazy initializer)
useState(JSON.parse(localStorage.getItem(key) || "null"))

// ✅ Lazy initializer — runs only once
useState(() => JSON.parse(localStorage.getItem(key) || "null"))
```

```
// ❌ Forgetting JSON.stringify — stores "[object Object]"
localStorage.setItem("user", user)

// ✅ Correct
localStorage.setItem("user", JSON.stringify(user))

// ❌ Forgetting JSON.parse — returns raw string "true" instead of boolean true
return localStorage.getItem(key) || initialValue

// ✅ Correct
return JSON.parse(localStorage.getItem(key)!)

// ❌ No try/catch — corrupted data crashes the app
return JSON.parse(localStorage.getItem(key)!)

// ✅ Wrapped in try/catch
try { return JSON.parse(...) } catch { return initialValue }

// ❌ Missing key in effect dependencies
useEffect(() => { localStorage.setItem(key, ...) }, [storedValue]) // key missing!

// ✅ Both dependencies
useEffect(() => { ... }, [key, storedValue])
```

Recap: `useLocalStorage`

Concept	What you learned
Lazy initializer	`useState(() => ...)` runs only once
JSON serialization	localStorage stores strings; use stringify/parse
Error handling	try/catch prevents crashes from corrupted data
Generics `<T>`	Makes the hook type-safe for any value type
`React.Dispatch<React.SetStateAction<T>>`	The exact type of a `useState` setter

---

3. `useDebounce`

Problem it solves (Without a Custom Hook)

Without debouncing, every keystroke triggers an API call:

```
function SearchPage() {
  const [searchTerm, setSearchTerm] = useState("");

  // Called on every keystroke
  useEffect(() => {
    fetchResults(searchTerm); // API call on every letter!
  }, [searchTerm]);

  return (
    <input
      value={searchTerm}
      onChange={(e) => setSearchTerm(e.target.value)}
      placeholder="Search..."
    />
  );
}
```

If the user types "React" quickly:

- `fetchResults("R")` — API call
- `fetchResults("Re")` — API call
- `fetchResults("Rea")` — API call
- `fetchResults("Reac")` — API call
- `fetchResults("React")` — API call

5 API calls for one search. The first 4 results are discarded. This is wasteful, potentially expensive, and can cause race conditions.

The fix: Only call the API when the user **stops typing** for a while (500ms). This is called **debouncing**.

The Concept

When to use it?

- Search inputs, autocomplete, any input that triggers expensive operations (API calls, filtering large lists)

When NOT to use it?

- Buttons — the user intends every click
- Form validation that should be immediate
- Real-time collaboration — needs instant updates

Step-by-Step Implementation

Step 1 — Understanding the Mechanism First

Before writing code, understand the JavaScript behind it.

`setTimeout` schedules a function to run after a delay:

```
const id = setTimeout(() => {
  console.log("Runs after 500ms");
}, 500);
```

`clearTimeout` cancels a scheduled timeout:

```
clearTimeout(id); // The function never runs
```

The debounce trick:

Every time the user types, we:

1. Cancel the previous timer (if any)
2. Start a new timer

Only the final timer (when the user stops typing) actually completes and updates the value.

Timeline with a 500ms delay:

```
0ms     User types "R"      → Timer A starts (fires at 500ms)
200ms   User types "Re"     → Timer A cancelled → Timer B starts
350ms   User types "Rea"    → Timer B cancelled → Timer C starts
500ms   User types "React"  → Timer C cancelled → Timer D starts
1000ms  No more typing      → Timer D fires → debouncedValue = "React"
                                            → 1 API call ✅
```

Step 2 — Simplest Version (No TypeScript)

```
// hooks/useDebounce.js
import { useState, useEffect } from "react";

function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timeoutId = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup: cancel the timeout if value or delay changes
    return () => {
      clearTimeout(timeoutId);
    };
  }, [value, delay]);

  return debouncedValue;
}
```

Every line explained:

`useState(value)` — stores the debounced value. Starts equal to `value`. Updates only after the delay.

`setTimeout(() => setDebouncedValue(value), delay)` — schedule an update after `delay` milliseconds. We save the ID to cancel it if needed.

`return () => { clearTimeout(timeoutId); }` — **This is the entire trick.** React runs this before the next effect execution. On every keystroke, the previous timer is cancelled before a new one starts.

`return debouncedValue` — returns a **single value**, not an array. Custom hooks can return anything.

Step 3 — The Cleanup Function — Deep Dive

This is the most important concept in `useDebounce`.

React runs the cleanup function before the next effect execution.

What happens WITHOUT `clearTimeout`:

```
Keystroke 1 → Effect runs → Timer 1 set
Keystroke 2 → Effect runs → Timer 2 set  (Timer 1 still running!)
Keystroke 3 → Effect runs → Timer 3 set  (Timers 1 & 2 still running!)
...
Timer 1 fires → setDebouncedValue("R")    → API call ❌
Timer 2 fires → setDebouncedValue("Re")   → API call ❌
Timer 3 fires → setDebouncedValue("Rea")  → API call ❌
Timer 4 fires → setDebouncedValue("Reac") → API call ❌
Timer 5 fires → setDebouncedValue("React")→ API call ✅
```

5 API calls. Debouncing failed completely.

With `clearTimeout` (correct):

```
Keystroke 1 → Effect runs → Timer 1 set
Keystroke 2 → Cleanup: clearTimeout(Timer 1) → Timer 2 set
Keystroke 3 → Cleanup: clearTimeout(Timer 2) → Timer 3 set
Keystroke 4 → Cleanup: clearTimeout(Timer 3) → Timer 4 set
Keystroke 5 → Cleanup: clearTimeout(Timer 4) → Timer 5 set

Timer 5 fires after 500ms → setDebouncedValue("React") → 1 API call ✅
```

> **The cleanup `return` is the entire mechanism. Without it, `useDebounce` is broken.**

Step 4 — Add TypeScript

```
// hooks/useDebounce.ts
import { useState, useEffect } from "react";

function useDebounce<T>(value: T, delay: number): T {
```

```
  const [debouncedValue, setDebouncedValue] = useState<T>(value);

  useEffect(() => {
    const timeoutId = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => {
      clearTimeout(timeoutId);
    };
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;
```

TypeScript explained:

- `<T>` — generic type variable. `T` is whatever type `value` is.
- `value: T` — the value being debounced. Its type is `T`.
- `delay: number` — always a number (milliseconds).
- `: T` — the return type. Same type as the input.

Why is the generic useful?

Without `<T>`:

```
function useDebounce(value: any, delay: number): any
// TypeScript can't help:
const debounced = useDebounce(searchTerm, 500);
debounced.toUpperCase(); // TypeScript doesn't know if this is valid!
```

With `<T>`:

```
const debounced = useDebounce("React", 500); // T = string
debounced.toUpperCase(); // TypeScript knows this is a string ✅
debounced + 1;           // TypeScript error: can't add string + number ✅
```

TypeScript infers `T` automatically:

```
useDebounce("React", 500);       // T = string
useDebounce(100, 500);           // T = number
useDebounce(true, 500);          // T = boolean
useDebounce({ id: 1 }, 500);    // T = { id: number }
```

Step 5 — How a Component Uses It

```
// components/SearchBox.tsx
import { useState, useEffect } from "react";
import useDebounce from "../hooks/useDebounce";

function SearchBox() {
  const [searchTerm, setSearchTerm] = useState("");
  const debouncedSearch = useDebounce(searchTerm, 500);

  // Only fires after user stops typing for 500ms
  useEffect(() => {
    if (debouncedSearch) {
      console.log(`API call with: ${debouncedSearch}`);
      // fetch(`/api/search?q=${debouncedSearch}`)
    }
  }, [debouncedSearch]);

  return (
    <div>
      <input
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        placeholder="Search..."
      />
      <p>Typing: {searchTerm}</p>
      <p>Searching for: {debouncedSearch}</p>
    </div>
  );
}
```

- `searchTerm` updates on every keystroke (immediate feedback in the input).
- `debouncedSearch` only updates after the user stops for 500ms.
- The API call uses `debouncedSearch`, not `searchTerm`.

Step 6 — Execution Flow

```
User types "React" (5 keystrokes)
        ↓
Each keystroke: setSearchTerm(newValue) → component re-renders
        ↓
Each re-render: useDebounce(searchTerm, 500) called
        ↓
useEffect dependency [value] changed → effect queued
        ↓
Before effect runs: cleanup → clearTimeout(previousTimer)
        ↓
Effect runs: new setTimeout(500ms) starts
        ↓
```

```
[repeats for each keystroke — all previous timers cancelled]
        ↓
User stops typing for 500ms
        ↓
Final timer fires: setDebouncedValue("React")
        ↓
debouncedValue state changes → component re-renders
        ↓
debouncedSearch = "React" → API called once
```

Usage Example

```
function SearchBox() {
  const [searchTerm, setSearchTerm] = useState("");
  const debouncedSearch = useDebounce(searchTerm, 500);

  // Only fires after user stops typing for 500ms
  useEffect(() => {
    if (debouncedSearch) {
      console.log(`API call: ${debouncedSearch}`);
      // fetch(`/api/search?q=${debouncedSearch}`)
    }
  }, [debouncedSearch]);

  return (
    <div>
      <input
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
        placeholder="Search..."
      />
      <p>Typing: {searchTerm}</p>
      <p>Searching for: {debouncedSearch}</p>
    </div>
  );
}
```

Common Mistakes — `useDebounce`

```
// ❌ Forgetting clearTimeout — timers pile up, debouncing fails entirely
useEffect(() => {
  setTimeout(() => setDebouncedValue(value), delay);
  // no return/cleanup!
}, [value, delay]);

// ❌ Timer outside useEffect — runs synchronously on every render
const id = setTimeout(() => setDebouncedValue(value), delay);
```

```
clearTimeout(id); // cancels it immediately — never fires

// ❌ Missing delay in dependencies — won't respond to delay changes
useEffect(() => { ... }, [value]); // delay is missing!

// ✅ Both dependencies
useEffect(() => { ... }, [value, delay]);

// ❌ Updating the debounced value immediately (no timer)
useEffect(() => {
  setDebouncedValue(value); // no setTimeout — no delay at all
}, [value]);
```

Recap: `useDebounce`

Concept	What you learned
Debouncing	Delay a value update until input stops changing
`setTimeout` / `clearTimeout`	Schedule and cancel timed operations
`useEffect` cleanup	Runs before next effect — the core mechanism
Dependency array	`[value, delay]` — re-run effect when either changes
Generic `<T>`	Makes the hook work with any type

Hook Comparison

Hook	Problem solved	React hooks used	Input	Output	Needs cleanup?	Generic?
`useToggle`	Boolean flip pattern	`useState`	`boolean?`	`[boolean, () => void]`	No	No
`useLocalStorage`	Persist state across refreshes	`useState`, `useEffect`	`string, T`	`[T, Dispatch<SetStateAction<T>>]`	No	**Yes**
`useDebounce`	Delay value updates	`useState`, `useEffect`	`T, number`	`T`	**Yes — critical**	**Yes**

- **Which hooks manage state?** All three — they all use `useState`.
- **Which hooks use effects?** `useLocalStorage` and `useDebounce`.
- **Which hooks use browser APIs?** `useLocalStorage` (localStorage API).

- **Which hooks require generics?** `useLocalStorage` and `useDebounce`.
- **Which hooks rely on cleanup?** `useDebounce` — cleanup is essential to its behavior.
- **Which hooks are most reusable?** All three — that's the point of extracting them.

React Concepts Learned

From `useToggle`

- Custom hook structure (function + `use` prefix)
- State is per-component, not per-hook-definition
- Functional state updates (`prev => !prev`)
- Tuple return types in TypeScript
- Array destructuring for flexible naming

From `useLocalStorage`

- Lazy state initialization (`useState(() => ...)`)
- Effects for side effects (writing to storage)
- Browser API interaction
- JSON serialization and deserialization
- Error handling in hooks
- Generic TypeScript types (`<T>`)
- `React.Dispatch<React.SetStateAction<T>>`

From `useDebounce`

- `useEffect` cleanup functions and when they run
- Timer management (`setTimeout` , `clearTimeout`)
- Why cleanup prevents stale timers
- The difference between a value and its debounced version
- Generics for flexible, type-safe hooks
- How `useEffect` re-runs and how to control it

Interview Questions

Basic

Q: What is a custom hook? A JavaScript function starting with `use` that can call other React hooks. Extracts reusable stateful logic from components. Not a special React API — just a naming convention.

Q: Why do custom hooks start with `use` ? React's ESLint plugin uses this to identify hooks and enforce the Rules of Hooks (no calling inside conditionals or loops). Without the prefix, violations go undetected.

Q: Can a custom hook contain other hooks? Yes. That's the entire point. `useLocalStorage` contains `useState` and `useEffect` . Custom hooks are compositions of existing hooks.

Q: What can a custom hook return? Anything — a single value, an array (tuple), an object, a function, or `undefined` . `useToggle` returns a tuple. `useDebounce` returns a single value. There is no restriction.

Intermediate

Q: Why use `prev => !prev` in `useToggle` instead of `!value`? `!value` captures `value` from the current render closure, which can be stale when React batches updates. `prev => !prev` always receives the latest queued state from React — always correct.

Q: Why use a lazy initializer in `useLocalStorage`? `localStorage.getItem()` is synchronous and runs on every render if passed directly. A function `useState(() => ...)` runs only on the first render, avoiding unnecessary reads.

Q: Why does `useLocalStorage` use `JSON.stringify` and `JSON.parse`? localStorage only stores strings. `JSON.stringify` converts any JavaScript value to a JSON string for storage. `JSON.parse` converts it back.

Q: Why should localStorage parsing use `try/catch`? `JSON.parse` throws `SyntaxError` on invalid input. Corrupted localStorage data (from browser extensions, manual edits) would crash the entire app without the catch. We catch and fall back to `initialValue`.

Q: Why is `useDebounce` implemented with `useEffect`? `setTimeout` is a side effect — it interacts with the browser's timer API outside React's rendering cycle. `useEffect` also provides the cleanup mechanism essential for cancelling previous timers.

Advanced

Q: Explain the cleanup mechanism in `useDebounce`. The cleanup function runs before each effect re-execution. Since the effect depends on `[value, delay]`, every value change triggers: cleanup → `clearTimeout(previous)` → new effect → new `setTimeout`. Only the last timer survives long enough to fire.

Q: What happens if `clearTimeout` is removed from `useDebounce`? Every keystroke's timer runs to completion. Typing 5 characters creates 5 timers that all fire — 5 state updates and 5 API calls. Debouncing fails entirely.

Q: How does TypeScript infer `T` in generic hooks? TypeScript reads the type of the argument and substitutes it for `T`. `useDebounce("React", 500)` → `T = string`. Writing `useDebounce<string>()` is redundant.

Q: What is the difference between `useState<T>` and `useLocalStorage<T>`? `useState<T>` stores the value only in React's in-memory state — lost on page refresh. `useLocalStorage<T>` has the same API but reads the initial value from `localStorage` and syncs every update to it, so the value persists across refreshes.

Q: How would you handle SSR (Next.js) in `useLocalStorage`? `localStorage` is only available in the browser. Guard against `window` being undefined on the server:

```
useState<T>(() => {
  if (typeof window === "undefined") return initialValue; // SSR guard
  try {
    const item = localStorage.getItem(key);
    return item ? JSON.parse(item) as T : initialValue;
  } catch {
    return initialValue;
  }
});
```

Q: What makes a custom hook well-designed?

1. Single responsibility — does one thing well
2. Mirrors existing React API conventions (like `useState`)
3. Properly handles errors and edge cases

4. Uses cleanup where needed
5. Is generic where the type can vary
6. Contains no JSX — pure logic only

Exercises

> Attempt each exercise before looking at solutions. The understanding you build by trying yourself is more valuable than reading a solution.

Exercise 1 — `useToggle`

Task: Create the `useToggle` hook and use it to build a modal component.

Expected behavior:

- A button says "Open Modal"
- Clicking it shows a modal overlay with some content and a "Close" button
- Clicking "Close" hides the modal
- The open/close state is managed entirely by your `useToggle` hook

Requirements:

- Create `hooks/useToggle.ts`
- Create `components/Modal.tsx` that uses it
- The hook must accept an optional initial value

Hints:

- Start with the hook. Test that it returns `[boolean, function]` .
- In the component: `const [isOpen, toggle] = useToggle()` .
- Use conditional rendering: `{isOpen && <div>...</div>}` .

Common mistakes to watch for:

- Using `setValue(!value)` instead of `setValue(prev => !prev)`
- Forgetting to export the hook
- Putting the toggle logic inside the component instead of the hook

---

Exercise 2 — `useLocalStorage`

Task: Build a settings page that persists username and theme preference.

Expected behavior:

- A text input for username, persisted under key `"username"`
- A toggle for dark/light theme, persisted under key `"theme"`
- Refresh the page — values should restore automatically
- Initial values: empty username, light theme (`false`)

Requirements:

- Create `hooks/useLocalStorage.ts` with generic `<T>`
- Create `components/Settings.tsx` that uses the hook twice
- Handle JSON errors with try/catch

Hints:

- `useLocalStorage<string>("username", "")` for the name
- `useLocalStorage<boolean>("theme", false)` for the theme
- The lazy initializer is crucial — `localStorage.getItem` must be inside a function passed to `useState`

Common mistakes to watch for:

- Forgetting `JSON.stringify` when writing
- Forgetting `JSON.parse` when reading
- Reading `localStorage` outside the lazy initializer (runs on every render)
- Missing `key` in the `useEffect` dependency array

Exercise 3 — `useDebounce`

Task: Build a search box that shows both the live input and the debounced value, and simulates an API call.

Expected behavior:

- As you type, the "Typing:" text updates immediately
- The "Searching for:" text updates only after 500ms of no typing
- When `debouncedSearch` updates, log `"API call: [value]"` to the console
- The console should show far fewer logs than keystrokes

Requirements:

- Create `hooks/useDebounce.ts` with generic `<T>`
- Create `components/SearchBox.tsx`
- Use two separate `useEffect` calls: one in the hook (for the timer), one in the component (for the "API call")

Hints:

- `searchTerm` from `useState` — updates on every keystroke
- `debouncedSearch` from `useDebounce(searchTerm, 500)` — updates after pause
- The input's `value` should be `searchTerm` (immediate)
- The API call `useEffect` depends on `[debouncedSearch]`, not `[searchTerm]`

Common mistakes to watch for:

- Forgetting `clearTimeout` in the cleanup return
- Using `searchTerm` (not `debouncedSearch`) as the dependency for the API effect
- Putting `setTimeout` outside of `useEffect`
- Not including `delay` in the `useEffect` dependency array